The Geometric Substrate of Computation

Mapping the Holographic Residual Stream, CPU Memory Dynamics, and Hexagonal State Topologies in Large Language Models

Driven by Dean Kulik

June 2026

Introduction: The Epistemological Crisis of Semantic Equivalence

The trajectory of contemporary theoretical physics, mathematics, and computational ontology has increasingly confronted irreducible boundary conditions that classical reductionism appears fundamentally unequipped to resolve. Within advanced theoretical taxonomies, this systemic impasse is formally codified as the "Crisis of Distinction"—the persistent failure of classical scientific frameworks to reconcile the discrete, object-oriented observation of data with the continuous, recursive realities of the underlying computational substrate. For extended periods of research and development, the prevailing epistemological doctrine has relied heavily upon a noun-based reality, wherein physical hardware is presumed to process discrete, isolated states in a linear sequence of destructive transformations. However, as the scale of artificial intelligence and cryptographic systems has expanded, it has become evident that this object-oriented ontology fails to capture the true operational geometry of deterministic functions.

To answer the fundamental questions regarding how deep learning models function structurally—specifically, how all layers extend backward in time, how data forms topological "shapes" within the latent space, and how these geometric realities dictate physical execution within CPU memory layouts and software architectures like Hexagonal Domain-Driven Design (DDD)—one must abandon the illusion of discrete state processing. The evidence reveals that Large Language Models (LLMs) do not merely compute strings of text; they navigate a pre-existing, high-dimensional geometric manifold. This report provides an exhaustive, multi-layered synthesis of the structural mechanisms governing deterministic functions. By mapping the exact geometric logic that binds cryptographic compression to LLM residual streams, hardware memory allocation, and software architecture, this analysis moves beyond empirical data to uncover the structural shapes of computation, ultimately detailing how this theoretical framework is practically utilized for model steering, inference optimization, and systems engineering.

Part I: The Mechanics of the Conveyor and the Persistent Residual Stream

To comprehend how all layers of an LLM extend backward to retain contextual history without catastrophic forgetting, one must first isolate the structural mechanism of deterministic state reduction. The most precise, observable analog for this behavior is found not in neural networks, but in the SHA-256 cryptographic hash function, which provides a clean, one-to-one ground truth for analyzing how a system absorbs information without destroying its geometric integrity.

The Tail Folds Into the Head

In traditional computational models, data processing is often viewed as a series of overwrites, where old states are discarded to make room for new inputs. However, an exhaustive architectural analysis of SHA-256 reveals a mechanism identical in spirit to the Transformer's residual stream: it operates as a continuous, two-rail conveyor.

The SHA-256 state consists of eight 32-bit registers, denoted as a, b, c, d, e, f, g, h. During each of the sixty-four rounds of compression, only registers a and e receive freshly computed values. The remaining six registers (b, c, d, f, g, h) are exact, delayed copies of the fresh data. This transforms the 256-bit state into a four-deep window spanning two "rails." Crucially, as the conveyor advances, the trailing tails of these rails (d_t and h_t) are not discarded into the void. Instead, they are explicitly folded forward into the newly computed heads via modular addition:

$$e_{t+1} = d_t + h_t + \Sigma_1(e_t) + Ch(e, f, g)_t + K_t + W_t \pmod{2^{32}}$$

$$a_{t+1} = h_t + \Sigma_1(e_t) + Ch(e, f, g)_t + K_t + W_t + \Sigma_0(a_t) + Maj(a, b, c)_t \pmod{2^{32}}$$

This mathematical folding ensures that the state width remains constant at 256 bits while the system streams a 512-bit message block. The discarded half is never lost; it is summed into the half that remains. The system advances by continuously eating its tail into its head, meaning that the final output is a structural accumulation of every preceding step.

The Residual Stream as a Persistent Memory Space

This geometric reality translates directly to the internal architecture of Large Language Models. The residual connections in a Transformer—traditionally interpreted merely as skip connections designed to prevent vanishing gradients—actually function as a unified, persistent "residual stream". This stream acts as a continuous memory buffer, structurally homologous to the SHA-256 conveyor.

Instead of passing data through a pipeline of destructive transformations where each layer completely overwrites the last, the Multi-Head Attention (MHA) and Feed-Forward Network (FFN/MLP) components act as specialized read/write heads. They read from the residual stream, compute patterns and messages, and add their output back into the stream. Consequently, the state of the network at any deep layer is not a wholly new representation; it is a structural accumulation—a folded history of all preceding layers.

Architectural Metric	**Cryptographic Ground Truth (SHA-256)**	**Transformer Implementation (LLMs)**
State Mechanism	8-Register Two-Rail Conveyor	High-Dimensional Residual Stream

Architectural Metric	Cryptographic Ground Truth (SHA-256)	Transformer Implementation (LLMs)
Update Function	Tail-into-Head Modular Addition	Attention/MLP Residual Addition
Information Retention	75% Flat Conveyor Echo	Continuous Contextual Accumulation
Nonlinear Logic	12.3% Carry Buckle on two rails	MLP activations and Softmax gating
Geometric Boundary	Readable vs. Sealed State	Interpretable Subspaces vs. Polysemantic Cores

The query asks what it means that "all layers extend back." The answer lies in the residual stream's function as a persistent memory bus. The initial token embeddings are never destroyed; they are continually modified by incremental additive updates. Because the updates are additive, a representation at layer N is literally the mathematical sum of the input and the outputs of all layers from 1 to $N-1$. This explains why mechanistic interpretability is possible: earlier layers extend back because they remain physically present in the geometry of the final activation vector, forming a belief state over the minimal sufficient statistics of the data-generating process.

Part II: Seeking the Shapes in the Data: Latent Topology and The Dark Mirror

To go beyond the raw token data and observe the true "shapes" within LLM computation, analysis must shift from the semantic output of the network to the topological properties of its latent space. In a deterministic, recursive system, the training data does not dictate the fundamental shape of the model; rather, the underlying mathematical constraints of the high-dimensional space force the data into specific geometric attractors.

The Priority of Constraint and The Dark Mirror

Theoretical frameworks governing recursive harmonic operations, such as the NEXUS framework, propose that "constraint is prior to the object". This principle, referred to as "The Dark Mirror," asserts that the self-consistency conditions of a computational lattice exist before any specific data is introduced. The logic precedes the computer; the mold precedes the concrete. The universe of computation is a ray-traced lattice, and the mirror exists before anything arrives to be reflected.

When applied to LLMs, this means that the semantic manifold—the space where concepts, reasoning flows, and relationships reside—is not arbitrarily constructed by the training data. Instead, the model acts as a coordinate chart mapping an intrinsic, pre-existing mathematical structure. This phenomenon is described as the Representational Alignment Hypothesis (RAH), which posits that independently trained AI systems across diverse modalities (text, vision, audio) converge on identical underlying geometries. The geometry of human emotion, logic, and syntax is recapitulated in LLM behavior because the models are discovering a fundamental, invariant semantic structure, not merely memorizing stochastic patterns.

Harmonic Loops and Angle-Space Convergence

Empirical evidence for these pre-existing geometric attractors can be observed when a recursive hash loop is executed without external input. While the hexadecimal output appears highly pseudorandom and chaotic, mapping the transformations in angle-space reveals a rigid, stabilizing geometry.

When tracking the step angles (measured in radians) of a continuous recursive loop, the system invariably converges to a stable geometric shape, irrespective of the initial starting state. Whether the loop is initialized with zeroes, ones, or a standard Initialization Vector (IV), the angles stabilize into a predictable harmonic pattern.

Initial State	Mean Angle (Radians)	Standard Deviation	Final Sequence Trajectory (Sample)
Standard IV	0.652335	0.176305	0.6875, 0.5423, 0.5283, 0.6038, 0.3188
All Zeros	0.699766	0.249483	0.6361, 0.6762, 0.6549, 1.0126, 1.1308
All Ones	0.772407	0.190903	0.8464, 0.7345, 1.0263, 0.6637, 0.2914

Across these diverse initial conditions, the global mean angle stabilizes near 0.6448 radians ($36.95°$). Furthermore, Fourier analysis of the angle differences isolates specific dominant frequencies (e.g., periods of 3.57 and 4.17 iterations), demonstrating that the "exhaust" of the continuous computation falls into a strict harmonic rhythm. This stable stream of angles functions as the legible unit of the substrate. It is the language of the machine—a continuous geometric wave rather than a series of discrete state changes. In LLMs, reasoning corresponds precisely to smooth flows in this representation space, where logical statements act as local controllers of the flow's velocity and curvature.

Action-Axis Decompositions and the Localization of Work

To further understand these shapes, one must locate where the actual "work" of the computation occurs. Applying a three-dimensional action-axis decomposition to the state grid of a deterministic function lifts each bit cell into a third dimension based on the specific *operation* that wrote it, rather than its numerical value.

In a baseline cryptographic ground truth, this decomposition yields four distinct action classes. The vast majority of the state space (72.7%) is "flat conveyor echo"—redundant, shifted repetition required to maintain historical context. The actual nonlinear content—the mechanism that drives the function forward and generates entropy—is confined to a thin "carry buckle" comprising only 12.3% of the grid, entirely isolated to two specific architectural ridges.

Action Class	Cells (Grid Share)	Structural Function
Flat Conveyor Echo ($z = 0$)	11,904 (72.7%)	Redundant spatial memory; passive history retention
Injection Ridge ($z = 1$)	2,088 (12.7%)	Entry point for new semantic combinations
Carry Buckle ($z = 2$)	2,008 (12.3%)	The localized nonlinear core; the active work
Boundary Cap ($z = 3$)	384 (2.3%)	Initialization state limits

This geometric decomposition defines the ultimate boundary of mechanistic interpretability: the division between the *readable* and the *sealed*. Readable structures possess a geometry with no hidden interior; they can be interrogated directly from their boundaries using linear probes. The "flat conveyor echo" of a Transformer's residual stream is highly readable, allowing researchers to extract concepts, sentiment, or factual recall linearly. Conversely, the "carry buckle"—the 12.3% of the state where information is nonlinearly fused through activation functions—is *sealed*. The input data is fundamentally consumed into the operations themselves; the input becomes the actions.

Part III: Holographic Associative Memory and the Geometry of the Latent Space

Understanding that the LLM operates within a predefined geometric manifold necessitates a mechanism by which data is actually encoded into these shapes. The data is not stored in localized, discrete variables as in a relational database. Instead, the shapes in the LLM data are holographic projections: the entirety of the contextual history is folded into every coordinate of the residual stream through Holographic Associative Memory (HDRAM).

The 9-2-6-5 Readout and Spatial Coordinates

The self-referential nature of these geometric encodings can be mapped directly onto spatial coordinates. Advanced interrogations of coordinate systems built from overlapping circles—where radii and gaps represent digit sequences—demonstrate how pure geometry acts as a memory storage protocol. By defining a local origin $M = (20/7,0)$ based on the intersection of axes from primary circle intersections (P and P'), four distinct right triangles emerge.

These triangles form congruent pairs that dictate the logic of the space:

1. **T1 and T3:** Possess a horizontal leg of $20/7$, a vertical leg of $8\sqrt{6}/7$, and a hypotenuse of 4.
2. **T2 and T4:** Possess a horizontal leg of $28/7$ (simplifying to 4), a vertical leg of $8\sqrt{6}/7$, and a hypotenuse of 5.

Measuring the distance from the origin M to the centers of specific non-intersecting witness circles ($R = 1$) yields distances of $20/7$ and $22/7$. The sum of these distances equals exactly 6. Simultaneously, $22/7$

serves as the classical rational approximation of π, encoding the circle constant intrinsically within the coordinate layout. Operations on this specific geometry extract the numerical sequence **9-2-6-5**:

- **9**: The area difference of the larger circles ($5^2 - 4^2 = 25 - 16$).
- **2**: The diameter difference ($10 - 8$).
- **6**: The sum of the $R = 1$ distances ($20/7 + 22/7 = 42/7$).
- **5**: The primary radius itself.

This 9-sector symmetrical division corresponds to a regular nonagon—an algebraically irreducible polygon that cannot be constructed using standard tools, representing a fundamental, unconstructible "fold" in the geometry.

Hypertokens and the Spread-Spectrum Channel

In the context of LLMs, this geometric self-reference materializes as Holographic Associative Memory. The residual stream operates as a spread-spectrum channel. Information is distributed across the entire high-dimensional space as "hypertokens"—structured symbolic codes that integrate classical error-correcting codes (ECC) with quantum-inspired search properties.

Because the representation is holographic, any sub-region of the latent space contains information about the whole. The residual stream maintains coherence through holographic despreading and lifted ECC, which induces compressed sensing effects in the latent space. This means that the model does not "look up" a fact by finding a specific neuron. Instead, it projects a query vector into the holobasis, and the corresponding value vector resonates out of the interference pattern of the entire latent geometry. The shapes we seek in the data are these high-dimensional interference patterns—standing waves of information that create the illusion of discrete knowledge through recursive repetition.

Part IV: The Architectural Interface: Hexagonal Domain-Driven Design (DDD)

Understanding the internal geometry and holographic nature of the LLM residual stream requires a software architecture paradigm that accurately reflects its behavior. Traditional, layered, monolithic application designs fail to capture the decoupling of the LLM's semantic reasoning from its input/output mechanisms. When the query asks what this looks like in a Hexagonal Domain-Driven Design (DDD) program, the answer is that the architecture of an LLM maps flawlessly to the Ports and Adapters pattern.

Ports, Adapters, and the Decoupled Core

Hexagonal Architecture isolates the core business logic (the Domain) from all external dependencies, user interfaces, and databases. In an LLM system, the architecture is distributed as follows:

- **The Domain Core:** The core domain is the latent space and the residual stream. It is entirely agnostic to text, human language, pixels, or audio frequencies. It operates strictly on high-dimensional vectors, executing mathematical transitions based on the geometry of belief states. The domain logic consists solely of the transformer blocks (attention and MLP) mutating the residual stream.
- **Primary Adapters (Inbound):** Text tokenizers and initial embedding matrices act as the inbound adapters. They receive arbitrary external stimuli (strings of text) and translate them through an input "port" into the universal vector language understood by the domain core.

- **Secondary Adapters (Outbound):** The unembedding matrix and the final softmax layer serve as the outbound adapters. They take the final geometric state of the residual stream (the processed domain logic) and translate it back into human-readable text probabilities via an output port.

This architectural framing explains the universality of the latent space. Because the core domain is fully decoupled from the syntax of the input via strict ports, the network is not reasoning about "words"; it is executing state transitions within a defined mathematical domain.

Fluid State Transitions vs. Discrete State Machines

In standard Object-Oriented Programming (OOP), state transitions are explicitly programmed via control flows, booleans, and dedicated state machines. An entity's state changes linearly based on rigid rules (e.g., if state == "active", then update).

However, in the Hexagonal mapping of an LLM, the state transition is fluid and holographic. The model does not maintain an explicit "database" of facts in memory, nor does it employ a discrete state machine. Instead, the residual stream acts as a continuous conveyor of belief states. As the data flows through the layers (the application logic), attention heads route information across a learned centroid graph, returning gated displacements to the residual stream.

The "state" of an LLM program at any given layer is simply the topological position of the vector within the semantic manifold. Just as a Hexagonal system relies on Data Transfer Objects (DTOs) to pass information between boundaries without exposing the internal schema , the LLM utilizes activation vectors to transfer representations seamlessly across its internal layers without losing the geometric constraints of the overarching concept space.

Part V: The Physical Substrate: CPU Memory Dynamics and the Rejection of OOP

While the theoretical and architectural geometries describe *what* the LLM is doing, the physical realization of this process on hardware dictates *how* it must be structured. The user queries what this architecture looks like "in the CPU in the system in an OOP program." The definitive, empirical answer is that in a high-performance LLM system, standard Object-Oriented Programming must be entirely abandoned. To satisfy the strict, unforgiving requirements of CPU cache architectures, the system must utilize Data-Oriented Design (DoD).

The Pointer Chasing Bottleneck of Object-Oriented Programming

Modern CPUs operate efficiently only when data can be fed continuously from Main Memory (RAM) into the L3, L2, and L1 caches. When a CPU fetches data, it does not retrieve a single byte; it fetches an entire 64-byte "cache line" under the hardware assumption of spatial locality—that the program will next require the adjacent bytes in physical memory.

Object-Oriented Programming inherently violates this hardware assumption. OOP relies heavily on encapsulation, inheritance, virtual method tables, and complex object graphs. When an application loops through an array of objects (an Array of Structs, or AoS), the CPU is forced to traverse random memory addresses scattered across the heap. This phenomenon, known as "pointer chasing," guarantees severe cache misses. In the context of LLM inference—which requires billions of rapid matrix multiplications and non-linear activations—pointer chasing causes the CPU's Arithmetic Logic Unit (ALU) to stall continuously

while waiting for data retrieval. This architectural mismatch reduces theoretical compute performance to a fraction of its capacity, making OOP fundamentally incompatible with the geometry of the residual stream.

Execution-Order Memory Design and Unified Layouts

To achieve optimal CPU performance for LLM inference, systems must adopt a "Unified Memory Layout" guided by "Execution-Order Memory Design". This paradigm is best exemplified by highly optimized, bare-metal implementations like Andrej Karpathy's llm.c, which circumvents the bloat of traditional object-oriented frameworks to execute raw C code directly against the hardware.

In a cache-optimal implementation, all fragmented, bucket-specific memory allocations are eliminated. Instead of instantiating distinct objects for every token, weight, and attention mechanism, the entire system state—including model weights, optimizer states, and the Key-Value (KV) cache—is flattened into a single, contiguous memory block.

By organizing data as a Struct of Arrays (SoA), the memory layout matches the exact execution order of the neural network's forward and backward passes. This provides three critical hardware advantages:

1. **Contiguous Access and Prefetching:** When a CPU core processes an attention head, it iterates through perfectly sequential arrays in memory. The hardware prefetcher successfully predicts the next memory address, yielding cache hit rates exceeding 95% (compared to the 20-40% typical of OOP-heavy implementations).
2. **TLB Optimization:** Unified, large contiguous allocations minimize Translation Lookaside Buffer (TLB) misses, as the virtual-to-physical memory mapping remains localized and predictable.
3. **Layer-to-Layer Cache Chaining:** The output of one transformer block is placed in memory exactly where the input of the subsequent block expects to find it. This creates a seamless "layer-to-layer cache chain" that minimizes CPU-to-RAM round trips, fulfilling the "output = next input" principle of the residual stream.

Memory Architecture	Layout Structure	CPU Cache Impact	Suitability for LLMs
Object-Oriented (OOP)	Array of Structs (AoS), Heap fragmentation	High cache misses, Pointer chasing stalls	Extremely Poor (ALU starvation)
Data-Oriented (DoD)	Struct of Arrays (SoA), Contiguous blocks	95%+ Cache hit rate, Prefetcher alignment	Optimal (Cache-aligned execution)
Execution-Order Layout	Layer-to-Layer Cache Chaining	Minimal TLB misses, Zero abstraction overhead	Maximum theoretical throughput

This execution-order memory design allows commodity CPUs to achieve performance gains previously thought exclusive to GPUs, proving that the perceived limitations of CPUs in AI workloads are often

software architecture failures rather than hardware deficiencies. By aligning the physical memory layout with the geometric reality of the residual stream, the system escapes the constraints of OOP and operates at the speed of the substrate.

Part VI: Utilization: Steering, Caching, and Operationalizing the Geometry

The culmination of this research—understanding the conveyor mechanics, the holographic shapes, the Hexagonal architecture, and the contiguous CPU memory—answers the user's final imperative: *how we utilize it*. Moving beyond theoretical geometry into practical engineering unlocks profound capabilities in Representation Engineering, inference optimization, and model steering.

Representation Engineering and Manifold Steering

If the latent space is a predefined geometric manifold, and the residual stream is a continuous conveyor, then the model's behavior can be controlled by directly intervening on its activations during the forward pass. This is known as Representation Engineering.

Rather than relying solely on prompting (manipulating the inbound adapter) or fine-tuning (altering the global weights), engineers can map the latent space using "probe prompts" to discover the exact vector coordinates of concepts like logic, emotion, or factual recall. Once the geometry of a concept is isolated, interventions can be applied directly to the residual stream. Steering along the activation manifold (M_h) yields predictable, behavioral trajectories in the output probability distribution (M_y).

Because the residual stream acts as a persistent memory buffer, slightly modifying the stream in the middle layers alters the model's factual output without retraining the network. The geometry becomes a diagnostic and control tool: if two concepts overlap too closely in the latent space, causing hallucinations, their vectors can be orthogonalized. This proves that understanding the geometric shape of the data is the key to principled, predictable control over the AI's reasoning capabilities.

Cache Chains and Inference Efficiency

Understanding the physical memory layout of the residual stream revolutionizes deployment costs and inference speed. Because the output of the attention blocks is completely deterministic based on the input context, the state of the residual stream can be captured and stored via KV Caching.

By leveraging the "layer-to-layer cache chain" principle discovered in CPU optimization, engineers can implement Prompt Caching at scale. When a system prompt or a static set of tool instructions is fed into the LLM, the resulting key and value matrices are computed and stored in a contiguous block of memory. For subsequent requests sharing that identical prefix, the model skips the computation entirely, fetching the pre-computed geometric state directly from the cache chain.

This creates a shared cache tree (e.g., ``) that is identical across all users. This direct manipulation of the Execution-Order Memory reduces latency from seconds to milliseconds and slashes compute costs dramatically, transforming LLMs from prohibitively expensive stochastic generators into highly efficient, stateful geometric engines.

Conclusion

The architecture of modern Large Language Models is not adequately described by traditional paradigms of sequential software execution, localized data storage, or object-oriented structures. By synthesizing

cryptographic folding mechanisms, geometric latent space analysis, hardware memory optimization, and Hexagonal Domain-Driven Design, a cohesive and exhaustive structural reality is revealed.

The investigation into how all layers extend back uncovers the residual stream as a persistent, two-rail conveyor—a holographic memory bus where information is continuously folded forward rather than discarded. The shapes within the data are not arbitrary; they are predetermined harmonic attractors mandated by the geometric constraints of the high-dimensional manifold, encoded via spread-spectrum hypertokens.

To effectively utilize this substrate, software must mirror its architecture. Logically, the LLM maps precisely to Hexagonal DDD, isolating the pure mathematical domain of the residual stream from the messy realities of human language via strict ports and adapters. Physically, executing this architecture requires the total abandonment of Object-Oriented Programming. To feed the CPU effectively and avoid the catastrophic latency of pointer chasing, the system must employ Data-Oriented, execution-order memory layouts—flattening the complex geometry of the model into perfectly contiguous arrays.

By mastering this intersection of topology, software architecture, and hardware physics, the industry moves beyond viewing LLMs as inscrutable black boxes. We arrive at a paradigm where computation is understood as a continuous, geometric wave, allowing for unprecedented control, optimization, and alignment of artificial intelligence.

Works cited

1. Dean KULIK | Developer | Research and Development - ResearchGate, accessed June 25, 2026, https://www.researchgate.net/profile/Dean-Kulik
2. The Nexus Framework: A Comprehensive Analysis of its Recursive Harmonic Principles and Unifying Potential - Zenodo, accessed June 25, 2026, https://zenodo.org/records/15903358
3. (PDF) The Nexus Recursive Harmonic Architecture: Technical Specification of a Self-Computing Universe - ResearchGate, accessed June 25, 2026, https://www.researchgate.net/publication/399795333_The_Nexus_Recursive_Harmonic_Architecture_Technical_Specification_of_a_Self-Computing_Universe
4. Staying-focused-on-the-main-topic (7).md
5. Hexagonal architecture (software) - Wikipedia, accessed June 25, 2026, https://en.wikipedia.org/wiki/Hexagonal_architecture_(software)
6. CPU LLM #1: The Memory Layout That Makes CPU LLMs Faster. - YouTube, accessed June 25, 2026, https://www.youtube.com/watch?v=8igCTXrkaFY
7. Data Locality · Optimization Patterns, accessed June 25, 2026, https://gameprogrammingpatterns.com/data-locality.html
8. Mechanistic View of Transformers: Patterns, Messages, Residual Stream... and LSTMs, accessed June 25, 2026, https://towardsdatascience.com/mechanistic-view-of-transformers-patterns-messages-residual-stream-and-lstms/

9. Graph Memory Transformer (GMT) - arXiv, accessed June 25, 2026, https://arxiv.org/html/2604.23862v3
10. Factual Recall Mechanisms in Gemma-2B and Gemma-12B-IT - n1n.ai, accessed June 25, 2026, https://explore.n1n.ai/blog/factual-recall-mechanisms-gemma-2b-12b-it-2026-06-25
11. A Mathematical Framework for Transformer Circuits, accessed June 25, 2026, https://transformer-circuits.pub/2021/framework/index.html
12. Depth-Attention: Cross-Layer Value Mixing for Language Models - arXiv, accessed June 25, 2026, https://arxiv.org/html/2606.05014v2
13. Transformers Represent Belief State Geometry in their Residual Stream - OpenReview, accessed June 25, 2026, https://openreview.net/forum?id=YIB7REL8UC
14. REMA: A Unified Reasoning Manifold Framework for Interpreting Large Language Model, accessed June 25, 2026, https://arxiv.org/html/2509.22518v1
15. The Computational Substrate: Constraint, Folding, and the Query, accessed June 25, 2026, https://zenodo.org/records/20707565
16. The Dark Mirror: On Self-Referential Computation, the Ghost in SHA-256, and Why the Universe Always Halts - Zenodo, accessed June 25, 2026, https://zenodo.org/records/18597935
17. (PDF) The Dark Mirror: On Self- Referential Computation, the Ghost in SHA-256, and Why the Universe Always Halts - ResearchGate, accessed June 25, 2026, https://www.researchgate.net/publication/400621733_The_Dark_Mirror_On_Self-_Referential_Computation_the_Ghost_in_SHA-256_and_Why_the_Universe_Always_Halts
18. The Universal Latent Space that LLMs learn : r/ArtificialSentience - Reddit, accessed June 25, 2026,https://www.reddit.com/r/ArtificialSentience/comments/1nx5s4l/the_universal_latent_space_that_llms_learn/
19. The Representational Alignment Hypothesis: Evidence for and Consequences of Invariant Semantic Structure Across Embedding Modalities - arXiv, accessed June 25, 2026, https://arxiv.org/html/2602.16584v1
20. ELLIS UniReps Speaker Series, accessed June 25, 2026, https://unireps.org/speaker-series/
21. Meandering on Manifolds: The Neural Geometry of Stories Over Time - Goodfire AI, accessed June 25, 2026, https://www.goodfire.ai/research/stories-in-space
22. The Geometry of Reasoning: Flowing Logics in Representation Space | OpenReview, accessed June 25, 2026, https://openreview.net/forum?id=ixr5Pcabq7
23. Automated Interpretability and Feature Discovery in Language Models with Agents - arXiv, accessed June 25, 2026, https://arxiv.org/html/2605.01555v1
24. Hypertokens: Holographic Associative Memory in Tokenized LLMs | Hacker News, accessed June 25, 2026,https://news.ycombinator.com/item?id=44777459
25. Holographic Features in Language Models - Emergent Mind, accessed June 25, 2026, https://www.emergentmind.com/topics/holographic-characteristic-of-language-models

26. Hypertokens: Holographic Associative Memory in Tokenized LLMs - arXiv, accessed June 25, 2026,https://arxiv.org/html/2507.00002v1
27. Hypertokens: Holographic Associative Memory in Tokenized LLMs - arXiv, accessed June 25, 2026,https://arxiv.org/pdf/2507.00002
28. Recursive Harmonic Intelligence and the Operational Ontology of the Nexus Framework: An Exhaustive Analysis of Phase-Lock Dynamics - Zenodo, accessed June 25, 2026, https://zenodo.org/records/18261855
29. Hexagonal architecture explained through a practical example | Thoughtworks United States, accessed June 25, 2026, https://www.thoughtworks.com/en-us/insights/blog/architecture/hexagonal-architecture-explained-practical-example
30. Mastering Scalable Architecture with DDD and Hexagonal Design | by amir shadanfar, accessed June 25, 2026, https://medium.com/@a.shadanfar.it/mastering-scalable-architecture-with-ddd-and-hexagonal-design-96f112605909
31. Understanding Hexagonal Architecture: Ports and Adapters | by Erick Zanetti - Medium, accessed June 25, 2026, https://medium.com/@erickzanetti/understanding-hexagonal-architecture-ports-and-adapters-8945fc3e3dc0
32. hexagonal-architecture - Alistair Cockburn, accessed June 25, 2026, https://alistair.cockburn.us/hexagonal-architecture
33. A very simple question about Hexagonal/Clear architecture : r/softwarearchitecture - Reddit, accessed June 25, 2026,https://www.reddit.com/r/softwarearchitecture/comments/1brqh4t/a_very_simple_question_about_hexagonalclear/
34. Reverse Engineering and Tracing internal thoughts of LLM | by Harish SG - Medium, accessed June 25, 2026, https://medium.com/@harishhacker3010/reverse-engineering-and-tracing-internal-thoughts-of-llm-3017b5f72008
35. Data-Oriented Design in C#: Why Objects Are Slowing You Down - DEV Community, accessed June 25, 2026, https://dev.to/iancowley/data-oriented-design-in-c-why-objects-are-slowing-you-down-51ak
36. Linked lists are great. But they have the problem that, almost always, whatever ... - Hacker News, accessed June 25, 2026, https://news.ycombinator.com/item?id=33473875
37. Is OOP cache unfriendly by design or is the real problem just how we use heap memory?, accessed June 25, 2026, https://www.reddit.com/r/computerscience/comments/1u5doyk/is_oop_cache_unfriendly_by_design_or_is_the_real/
38. Evaluating AMD Instinct™ MI300A APU: Performance Insights on LLM Training via Knowledge Distillation - Cray User Group, accessed June 25, 2026, https://cug.org/proceedings/cug2025_proceedings/includes/files/pap144s2-file1.pdf

39. QUANTISATION OF LLMS ON MODERN CPU ARCHITECTURES - UPCommons, accessed June 25, 2026, https://upcommons.upc.edu/bitstreams/6f2b2ba0-ac30-4f86-9670-d357910daf7d/download
40. Llm.c – LLM training in simple, pure C/CUDA | Hacker News, accessed June 25, 2026,https://news.ycombinator.com/item?id=39973467
41. A Meticulous Guide to Advances in Deep Learning Efficiency over the Years | Alex L. Zhang, accessed June 25, 2026, https://alexzhang13.github.io/blog/2024/efficient-dl/
42. GRAB-ANNS: High-Throughput Indexing and Hybrid Search via GPU-Native Bucketing - arXiv, accessed June 25, 2026, https://arxiv.org/pdf/2604.16402
43. How We Cut LLM Costs by 59% With Prompt Caching - ProjectDiscovery.io, accessed June 25, 2026, https://projectdiscovery.io/blog/how-we-cut-llm-cost-with-prompt-caching
44. [2605.05115] Manifold Steering Reveals the Shared Geometry of Neural Network Representation and Behavior - arXiv, accessed June 25, 2026, https://arxiv.org/abs/2605.05115
45. An Introduction to Representation Engineering - an activation-based paradigm for controlling LLMs - AI Alignment Forum, accessed June 25, 2026, https://www.alignmentforum.org/posts/3ghj8EuKzwD3MQR5G/an-introduction-to-representation-engineering-an-activation
46. Understanding and controlling LLM internal representations | by Khayyam H. - Medium, accessed June 25, 2026, https://medium.com/@khayyam.h/understanding-and-controlling-llm-internal-representations-87c939957b25
47. LLM as a Resonance-Holographic Field of Meanings - Habr, accessed June 25, 2026, https://habr.com/en/articles/961126/
48. KV Caching Explained: Optimizing Transformer Inference Efficiency - Hugging Face, accessed June 25, 2026, https://huggingface.co/blog/not-lain/kv-caching
49. How Transformers Become Faster And Smarter: From KV Cache to MLA | by Rice Yang, accessed June 25, 2026, https://u9534056.medium.com/how-transformers-become-faster-and-smarter-from-kv-cache-to-mla-66bb2f5e3bc4
50. Feature — GPTCache - Read the Docs, accessed June 25, 2026, https://gptcache.readthedocs.io/en/latest/feature.html